



Contents lists available at ScienceDirect

Journal of Information and Intelligence

journal homepage: www.keaipublishing.com/en/journals/journal-of-information-and-intelligence

DelRightGuard: A secure yet lightweight data deletion notification distribution protocol for safeguarding right to deletion

Qipeng Song, Ruiyun Wang, Yue Li^{*}, Yiheng Yan, Xingyue Zhu, Hui Li

School of Cyber Engineering, Xidian University, Xi'an 710071, China

ARTICLE INFO

Keywords:

Right to deletion
Deletion notification distribution and acknowledgement
Privacy protection

ABSTRACT

In recent years, the right to deletion of individual has been recognized by many privacy protection laws and regulations. It stipulates that the data controller receiving individual data deletion request shall not only erase the required data, but also take reasonable steps to inform other data controllers to delete the same data. Prior to irrecoverable data erasure, it is of paramount importance to design a distribution and acknowledgement process of deletion notifications across involved data controllers. The design of such a mechanism is faced with the following challenges: 1) completeness: Ensuring that all relevant data controllers, who possess the data slated for erasure, are duly informed; 2) robustness: Immune from malicious attacks when deletion notifications traverse through untrusted networks; 3) lightweight: Reduce the right to deletion compliance cost and accommodate more deletion requests for data controllers. To this end, this article proposes DelRightGuard, which is the first attempt to tackle with the aforementioned challenges. DelRightGuard is built on a cross-plane cooperation architecture between regulatory and service planes. Within regulatory plane, DelRightGuard proposes a cuckoo filters based on data circulation recording algorithm to efficiently ensure the completeness of deletion notifications. Within service plane, DelRightGuard devises a secure yet lightweight deletion notification distribution protocol that runs on a network function hosted by each data controller. This protocol employs HMAC based hop-by-hop forward traversal verification, recursive backward acknowledgement and probabilistic sampling verification, so that it ensure the robustness and lightweight of deletion notification distribution process. We implement a prototype for DelRightGuard. The experimental result confirms that it is practical with acceptable performance.

1. Introduction

The rapid development of data-driven services and the rise of data spaces—large-scale, cross-domain ecosystems for regulated data sharing—have brought new challenges to privacy protection. Within such environments, personal data often circulates among multiple autonomous data controllers. Consequently, the enforcement of the “right to be forgotten” (also known as the right to deletion or erasure) has become more complex and critical. Laws such as the European Union's General Data Protection Regulation (GDPR) [1], California's Consumer Privacy Act (CCPA) [2], and China's Personal Information Protection Law [3] mandate that data controllers (e.g., companies, platforms, institutions) must not only delete personal data upon request but also take reasonable steps to notify other controllers to do the same [4].

^{*} Corresponding author.

E-mail addresses: qpsong@xidian.edu.cn (Q. Song), 23151214318@stu.xidian.edu.cn (R. Wang), liyue@xidian.edu.cn (Y. Li), 20009201390@stu.xidian.edu.cn (Y. Yan), zhuxingyue@stu.xidian.edu.cn (X. Zhu), lihui@mail.xidian.edu.cn (H. Li).

<https://doi.org/10.1016/j.jiixd.2024.11.001>

Received 16 April 2024; Received in revised form 25 September 2024; Accepted 6 November 2024

Available online 3 April 2025

2949-7159/© 2025 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Despite these legal obligations, the technical mechanisms needed to guarantee this right remain immature. Enforcing the right to deletion involves solving two core challenges: 1) *verifiable and irreversible data deletion*: Ensuring that the target data is permanently deleted by the data controller; and 2) *secure deletion notification cross-domain distribution*: Ensuring that all involved data controllers are reliably notified of the deletion request¹. Prior research has predominantly focused on ensuring *verifiable and irreversible data deletion* within a single data controller [5–8]. However, far less attention has been paid to the equally important task of *securely distributing deletion notifications* across multiple, potentially untrusted, data controllers—a gap that introduces significant compliance risks, especially in distributed data-sharing ecosystems like data spaces. This paper addresses this overlooked yet foundational issue.

In data spaces, personal data may traverse diverse domains and jurisdictions. To ensure meaningful deletion, a deletion request must propagate across the entire historical data-sharing path. In the absence of a secure, traceable, and efficient distribution mechanism, deletion notifications may be lost, spoofed, misdirected, or dropped—ultimately undermining user privacy and regulatory compliance (see Section 3.2 for threat details).

Designing such a mechanism is non-trivial. First, guaranteeing that all relevant data controllers are notified requires tracking the full data circulation history across domains, which is expensive and burdensome if each controller manages its own logs. Second, as privacy awareness rises and deletion requests increase, any notification mechanism must be lightweight and cost-effective to ensure widespread adoption. Since complying with deletion requests offers no direct economic benefit to data controllers, minimizing operational overhead is crucial.

To meet these challenges, we model the distribution process of deletion notification as a spanning-tree rooted at the data controller that receives the user's deletion request. This enables formalizing deletion propagation as a bidirectional traversal verification problem over the data-sharing graph. We thus propose a novel two-plane architecture, DelRightGuard, which separates concerns between a regulatory plane and a service plane. The regulatory plane, overseen by a central regulatory body, maintains a compact and efficient record of data circulation history using a privacy-preserving structure based on cuckoo filters. The service plane, composed of participating data controllers, executes a secure and verifiable deletion notification distribution protocol. This protocol incorporates trusted execution environments (TEEs), message authentication codes, recursive acknowledgments, and adaptive probabilistic sampling to ensure both robustness and scalability.

- (1) *Formulation of secure deletion notification distribution*. We are the first to point out the importance of secure deletion notification distribution over untrusted underlying network and formulate it as a spanning-tree bidirectional traversal verification problem. The deletion notification distribution is reasonably modeled as a spanning-tree like procedure, same to the one representing the circulation history of data to be deleted. Thus, to ensure the secure distribution of deletion notifications can be reduced to how to verify if the actual distribution procedure is compliant with the expected one.
- (2) *Cross-layer cooperation centralized network architecture*. We propose a cross-layer cooperation centralized network architecture supporting the secure distribution of deletion notifications. It consists of regulatory plane and service plane. Regulatory plane mainly includes a central regulatory agency responsible for recording data circulation history and accordingly construct data circulation graph upon query from service plane, while service plane incorporates a large number of data controllers willing to operate in right-to-be-forgotten compliant way. The proposed architecture is capable of enabling regulatory agency and data controllers to collaboratively solve the problem of secure distribution of deletion notification at their own reasonable expense.
- (3) *Cuckoo filters based data circulation recording at regulatory plane*. We propose an efficient data circulation path storage mechanism aimed at ensuring the completeness and trustworthiness of deletion instruction notifications. Based on the cuckoo filter, we design a novel data structure that supports efficient insertion of data circulation history, precise querying of the previous hop in the data circulation path, and accurate deletion of the data circulation history. This mechanism dynamically adapts to increasing data volumes, ensuring scalability and long-term operational stability, enabling efficient processing and precise tracking of the circulation history of large-scale data. Furthermore, the mechanism accurately records data circulation history and reconstructs the complete circulation path of the data to be deleted, addressing the limitation of traditional cuckoo filters, which cannot directly provide mappings of relationships between data elements.
- (4) *A Lightweight yet secure deletion notification protocol at service plane*. We design a deletion right guard (DelRightGuard) for data controllers within service plane. Enabled by trusted execution environment (TEE, such as Intel's SGX, ARM's TrustZone), DelRightGuard is mainly built on the basis of message authentication code. It employs the hop-by-hop forward traversal verification, recursive backward acknowledgement and probabilistic sampling verification, so that it can deal with arbitrary topology built from circulation history of target data. With DelRightGuard, the involved data controllers can securely distribute deletion notifications, immune from the potential malicious attacks in the threat model, in a lightweight and robust manner.

2. Related work

In recent years, as citizens become increasingly aware of their right to deletion, research about right to deletion protection has attracted lots of attention. The existing studies can be broadly classified into the following categories: irrecoverable data deletion, verifiable data deletion, and machine unlearning.

¹ We distinguish between a *deletion request*, initiated by a user toward a data controller, and a *deletion notification*, forwarded among data controllers.

2.1. Irrecoverable data deletion

Irrecoverable data deletion focuses on methods for permanently erasing data from storage devices, ensuring that the data cannot be recovered. Numerous techniques, such as cryptographic key deletion, secure overwriting, and hardware-based self-destruction mechanisms, have been proposed in the literature. These methods mainly address the challenge of making data unrecoverable within a single data controller but do not address the issue of secure deletion notification distribution across multiple data controllers, which is the focus of our research.

Numerous researchers have explored methods for permanently and irrecoverably erasing data from physical media. Reardon et al. [9] investigate the approaches for irrecoverable data deletion at different interfaces to physical medium, and present a taxonomy of adversaries with different capabilities. Lee et al. [10] introduce a secure deletion technology for USB flash storage, which requires only a single write or erase operation, while offering improved performance. Mo et al. [11] propose a secure deletion scheme for cloud storage data, achieving fine-grained data deletion through key modulation techniques, avoiding reliance on third-party servers, and simplifying key management. Xiong et al. [12] propose a secure self-destruction scheme for encrypted data in cloud, on the basis of key-policy attribute-based encryption technology with time attributes (KP-TSABE). The sensitive data will be securely self-destructed after a user-specified expiration time. Yang et al. [13] propose a secure user space data deletion method for Android. They provide secure deletion at the file level through the use of user-space file systems, independent of the underlying file system and storage medium characteristics. Xiong et al. [5] present a novel key derivation encryption (KDE) algorithm and accordingly construct a secure data deletion (SDDK) scheme for encrypted data deletion on IoT devices. Gnatyuk et al. [14] survey data deletion standards in different countries and propose a randomized overwriting-based method STM Shredder for guaranteed data deletion. Data sanitizing cause various data “remnant” on the flash storage medium. These remnants may be utilized to recover the deleted data. RedFlash [15] is proposed to remove both the data and the corresponding duplicates to ensure the irrecoverable deletion on flash memory, at the cost of a small performance degradation.

2.2. Verifiable data deletion

Verifiable data deletion aims to validate for the data owner that the required data has been conclusively erased. However, these methods focus solely on ensuring deletion within individual data controllers and do not provide solutions for distributing deletion notifications across different controllers in large-scale, cross-domain environments.

The principal methodologies in verifiable deletion research encompass strategies such as encryption key deletion, overwriting techniques and blockchain technology.

The core idea of encryption key deletion is to permanently get rid of the key for encrypting data pending deletion so that the encrypted data cannot be accessed. In this regard, Xue et al. [16] propose a reliable encrypted data deletion scheme for cloud data, based on attribute-based key policy encryption (AD-KP-ABE). This approach not only facilitates fine-grained data access but also ensures the reliable deletion of data. Zheng et al. [17] introduce a trusted deletion system model for cloud storage, incorporating techniques for verifiable deletion and proposing various methods for timely and fine-grained deletion verification. Ma et al. [18] propose a scheme using ciphertext-policy attribute-based encryption (CP-ABE) technology, achieving refined secure deletion of cloud data through the construction of an attribute association tree. Furthermore, the scheme introduces a rule substitution algorithm, enhancing the verification capability of the deletion process, and improving the security and reliability of data deletion verification. Tian et al. [19] present a strategy that integrates multi-level security principles and dynamic technologies to refine access control, effectively manage data deletion, and provide verifiable proof of deletion using a Merkle Hash tree.

In the study of overwriting based deletion verification, the data owner generates random data blocks and send to the server. The server overwrites data to be deleted with these random blocks and generates a verification value for each data block after the overwrite. For example, Bauer et al. [20] present an implementation of the Ext2 file system that, by using asynchronous overwriting, sacrificed immediate security but ultimately achieved a more useable and complete secure deletion. Bhat et al. [21] propose a reliable and efficient stackable file system, restFS, to address the shortcomings in reliability and efficiency of existing transparent secure file deletion system extensions. Unlike the file-level overwriting found in existing technologies, restFS improves reliability and efficiency through block-level overwriting. Du et al. [22] propose a scheme based on ciphertext re-encryption and overwrite verification (WV-CP-ABE) that performs refined deletion of data by changing the ciphertext of the access control policy, and verifies the correctness of the data to be deleted using a searchable path hash binary tree (DSMHT) based on dirty data block overwriting. Yang et al. [23] propose an efficient and fine-grained outsourced data deletion scheme based on reversible Bloom filters, which not only enables the public and private verifiability of storage and deletion results but also allows users to easily detect malicious behavior of the cloud server with a high probability when it dishonestly maintains/deletes data and generates corresponding evidence.

Blockchain technology emerges as a promising solution to enable verifiable data deletion, offering a means to guarantee the efficacy and traceability of deletion operations without dependence on any trusted third-party entity. For instance, Yang et al. [24] introduce a blockchain-based scheme that allows for the public verification of personal information deletion. Within this framework, following a data deletion operation, a deletion credential is generated and recorded on the blockchain, thereby ensuring the verifiability of data deletion.

2.3. Machine unlearning

Machine unlearning focuses on removing specific data points from machine learning models, ensuring that these points no longer influence the model's output. However, it does not address the need for secure deletion notification distribution across multiple data

controllers in a cross-domain setting, which is the core focus of DelRightGuard.

Another avenue of research focuses on the derivative impact of data deletion, such as machine unlearning. Machine unlearning aims to remove points from the training dataset of a machine learning model after training [25–27]. SISA [28] is a computational efficient framework to facilitate the machine unlearning process by partitioning data and thus reducing a single data point's influence on the training of machine learning models. Sekhari et al. [29] concentrate on exact forgetting, providing an efficient machine unlearning algorithm in terms of computational and storage complexity. BAERASER [30] is a backdoor defense method against backdoor injection attacks in neural networks, utilizing machine unlearning techniques. It first recovers trigger patterns from the victim model using an entropy maximization-based generative model, then employs a gradient ascent-based machine unlearning method to reverse the injection process and erase the model's backdoor traces without requiring full access to the training data.

Some research efforts care about how to verify the effectiveness of machine unlearning. Sommer et al. [31] introduce a formal framework for designing verification mechanisms for machine unlearning. Grounded in standard hypothesis testing for rigorous quantification, the work unveils a novel backdoor-based verification approach to ensure high-confidence data deletion. Eisenhofer et al. [32] introduce the first cryptographic framework for verifiable machine unlearning, addressing the lack of user-auditable methods to ensure data is effectively removed from machine learning models. Leveraging SNARKs and Merkle trees, the framework offers cryptographic proofs of data update and unlearning.

Despite intention of machine unlearning to protect privacy, machine unlearning may inadvertently leave traces of data in the ML model, leading to unintended privacy breach. Chen et al. [25] introduce a novel membership inference attack leveraging the output differences between the original ML model and its unlearned version to identify if a specific sample was part of the original training set but not in the unlearned model's training set. The study also explores mitigation mechanisms for these privacy risks.

2.4. Discussion

The right to data deletion presents two major challenges: 1) the irreversible deletion of data within a single data controller, and 2) the secure distribution of deletion notifications to all relevant data controllers. While the first challenge has been extensively addressed through techniques such as secure deletion, verifiable deletion, and machine unlearning, the second remains largely under-explored—particularly in the context of cross-domain data-sharing ecosystems.

This gap is critical: Even if one data controller complies with a deletion request, others in the data-sharing chain may retain the data unknowingly, resulting in compliance failures and privacy risks. Existing work rarely considers the need for a coordinated, secure notification process among controllers. DelRightGuard explicitly addresses this overlooked challenge. Rather than focusing solely on data deletion mechanisms, it approaches the problem as one of secure and verifiable notification distribution—ensuring that all relevant data controllers are reliably informed of deletion requests, even in untrusted network environments.

3. Problem statement

3.1. How does deletion notification distribution work?

Although the right to erasure mandates that, when citizens exercise this right, all data controllers sharing the target data are obliged to execute reasonable deletion actions, current laws and regulations fail to specify how all involved data controllers should be duly notified. Both the industrial and academic sectors have yet to propose explanations for this procedural gap. Intuitively, one might assume that the process of distributing deletion notifications among data controllers opts for a straightforward “one-to-many” model illustrated in Fig. 1(a), wherein the first data controller (referred to as the “source data controller”) to receive the deletion request notifies all other data controllers holding the data to be deleted.

However, we argue that this approach imposes an unreasonable performance burden on the source data controller. For instance, consider data controller A, who shares certain data with data controller B, who in turn shares it with 1000 other controllers. When a user contacts the source data controller to delete this data, it would be obligated to notify the other 1001 data controllers to erase the target data. This clearly seems unfair to A, as it almost singularly bears the cost of compliance with the right to erasure, while other data controllers enjoy the benefits of shared data with minimal obligations. Such a distribution model is unlikely to be acceptable to data controllers.

In light of this, it is more reasonable to assume that the distribution model of deletion notifications among data controllers follows the model illustrated in Fig. 1(b). The model uses the source data controller as the root node, distributing to all successor data controllers who have shared the data to be deleted, until the final leaf nodes are notified. The paths of deletion notifications ultimately forms a spanning tree with the source data controller at the root, encompassing all relevant data controllers. This spanning tree aligns with the historical circulation path of the data pending deletion. In this model, each data controller reasonably participates into the distribution process of the deletion notifications. Compared to the one-to-many model, this model reduces the computational and network resource consumption caused by compliance with the right to erasure at the source node and embodies the legal principle of “He who benefits, bears the responsibility”.

3.2. Threat model

We reasonably assume that the data controllers have strong incentives to protect the right to deletion of their clients. The reason are twofold: One hand, if a data controller illegally retains data that should be deleted, it will face financial penalties and even legal proceedings; on the other hand, there are now more and more technologies that can allow users to verify whether a data controller has

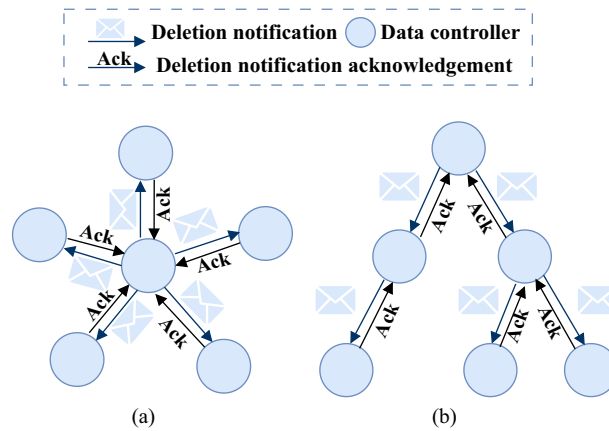


Fig. 1. Distribution model of deletion notifications among data controllers. (a) One-to-many distribution model. (b) Spanning-tree distribution model.

actually deleted their personal data (e.g., machine unlearning [7], verifiable data deletion [6], etc.).

For the scenario of cross data controllers data sharing, the enforcement of deletion right involves a large number of data controllers situated in different regions. When deletion notifications are distributed through the untrusted network, it is possible for adversaries to disturb the distribution process of deletion notification for a variety of purposes. The distribution of deletion notifications traversing untrusted networks is susceptible to security vulnerabilities as follows, shown in Fig. 2.

- (1) **Tampering of deletion notification:** The tampering of deletion notification attack consists of unauthorized modification, alteration of the content of a deletion notification, resulting in a change in the user's intent to delete (refer to Fig. 2(a)). For example, Alice intends to delete her personal data in a data controller, the data controller sends a deletion notification, and after intercepting Alice's deletion request, the malicious attacker Eve modifies Alice's identity into that of Bob, which ultimately leads to the deletion of Bob's information.
- (2) **Impersonation:** Impersonation refers to the scenario where Eve disguises as the previous hop data controller, deceiving it into executing malicious deletion notification without authorization (refer to Fig. 2(b)). For instance, the malicious attacker Eve obtains Alice's ID then masquerades as a data controller in Alice's personal information circulation tree, and sends a false deletion notification to the target data controller holding Alice's private information. This results in the data controller deleting Alice's personal privacy information without her authorization, not only affecting Alice's normal services with the data controller but also severely disrupting the data controller's regular operations.
- (3) **Path deviation:** Path deviation occurs when Eve redirects a deletion notification to an unrelated data controller by changing the destination address (refer to Fig. 2(c)). This misdirection causes the wrong data controller to delete user data erroneously, while the intended recipient fails to receive and process the notification, leading to unwanted data retention. For instance, Eve intercepts a deletion notification for Alice from data controller M to N, rerouting it to data controller P. As a result, P deletes Alice's data, while N, the intended recipient, doesn't process the notification, adversely affecting Alice's services at N and posing significant risks for both users and data controllers.
- (4) **Replay attack:** Replay attack occurs when Eve captures and saves a previously received legitimate deletion notification from a data controller. Later, Eve resends this notification to trick the data controller into thinking it's a valid user request (refer to Fig. 2(d)). For example, Eve intercepts and stores a notification to delete Alice's data sent from one data controller to another. Later, Alice resends it to the target data controller, making it appear as if Alice resubmitted the notification. However, Alice had uploaded new data before Eve's replay. The target data controller, treating the replayed notification as legitimate, deletes Alice's new data by mistake.
- (5) **Drop attack:** In a drop attack, Eve intercepts a deletion notification from one data controller to another, discards it, and sends a forged acknowledgement to the previous hop data controller (refer to Fig. 2(e)). This deception aims to penalize the target data controller for non-compliance with deletion regulations. For example, Eve intercepts and discards a deletion notification from data controller M to N for Alice's data, then falsely informs M of its success. Consequently, M remains unaware of the unexecuted deletion, facing penalties, while Alice's data is not deleted as intended.

We assume that the attacker cannot observe or modify any data or program code running within an enclave, and the enclave is trusted to attest to the integrity of the code running therein. While existing enclave solutions—such as SGX, which we build on—fall short of meeting the abstract enclave assumption perfectly. The security issue of current enclave solutions is out of scope for this work, as such fixes are an active area of research in their own right [33,34].

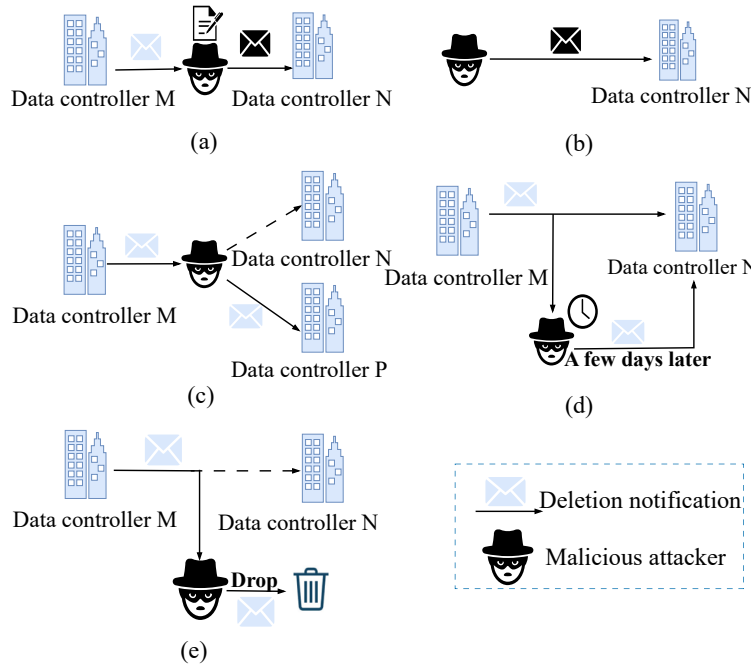


Fig. 2. Threat model. (a) Tampering of deletion notification. (b) Impersonation. (c) Path deviation. (d) Replay attack. (e) Drop attack.

3.3. Design goals

The design of DelRightGuard focuses on three key objectives: completeness, security, and lightweight implementation. Completeness ensures that all relevant data controllers receive deletion notifications, preserving individuals' right to deletion without omissions. This is achieved through a structured "deletion notification propagation tree", ensuring notifications are accurately distributed from source to all endpoints. Security is critical for protecting notifications in untrusted networks, safeguarding the process against potential attacks as outlined in the threat model, even under powerful adversaries. Lightweight implementation minimizes operational overhead, encouraging adoption by data controllers, as compliance with the right to deletion typically provides no direct economic benefit. By maintaining low resource demands, DelRightGuard offers a practical, secure, and efficient solution for deletion notification distribution.

4. Design overview

This section provides an intuitive overview of the key components of DelRightGuard, each of which is detailed in Section 5.

4.1. Cross-layer cooperation centralized management architecture

We propose a centralized management architecture with cross-layer collaboration to ensure the secure and trustworthy distribution of deletion notifications among data controllers. As shown in Fig. 3, the management architecture consists of regulatory and service planes. The regulatory plane is primarily composed of central regulatory agency. Its responsibility is to record the history of data circulation among data controllers. The service plane is made up of numerous data controllers who, in addition to normal business activities, should report information related to data circulation towards the regulatory agency, before sharing data with other data controllers. In case of data deletion, the data controller receiving deletion request queries the graph that depicts the whole data circulation process from the regulatory plane. The former then constructs the deletion notification and initiate the notification distribution on the service plane. At the end, it receives an acknowledgement to determine if all the relevant data controllers are correctly notified.

One may ask why a centralized management architecture is adopted in favor of a distributed one. Centralized management of data circulation history has advantages in providing transparency in circulation, enhancing the trust from data subject (i.e. individual), and complying with specific regulatory requirements. The participation of a trusted regulatory agency offers an objective recording and verification mechanism for data circulation, which can be very useful in dispute resolution and legal audits. Distributed solutions where data controllers locally maintain their own data circulation history, increase the compliance operational costs for data controllers and require organizations to have robust internal control and audit processes to ensure the accuracy and integrity of records, and to provide necessary information during regulatory reviews. Furthermore, some privacy protection laws and regulations stipulate that in certain

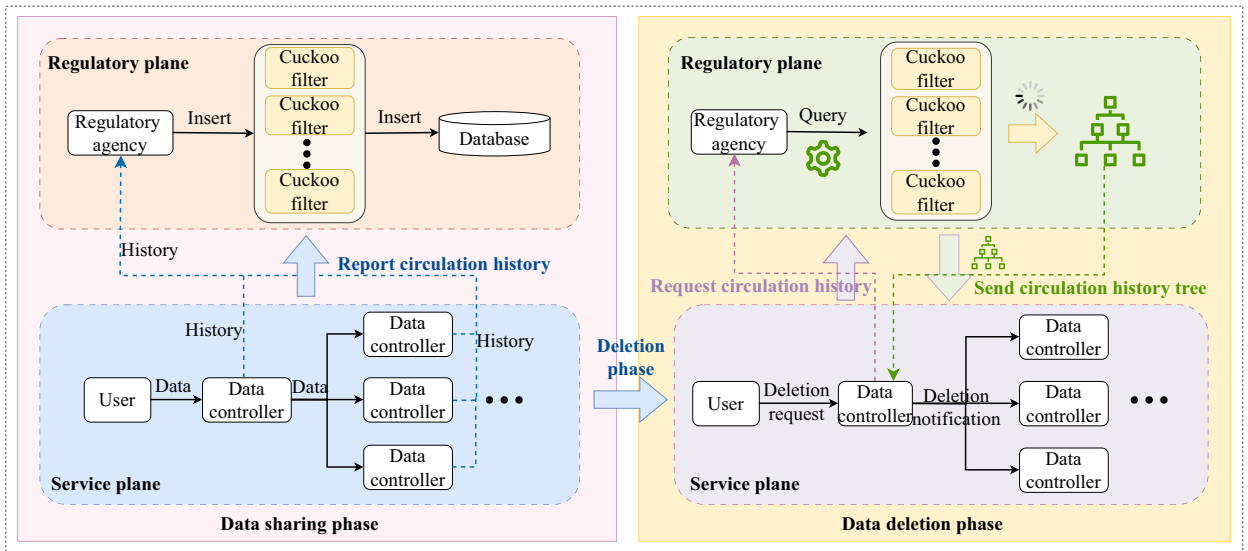


Fig. 3. Overview of the workflow for DelRightGuard.

scenarios, particularly concerning sensitive personal data (such as health information, financial information, etc.), it is required to obtain prior permission from regulatory bodies before sharing. Therefore, this article suggests adopting a cross-layer collaborative centralized management architecture, where the regulatory and service planes coordinate at mutually acceptable costs, to ensure the protection of the right to deletion.

4.2. Cuckoo filters based data circulation recording

One of the key challenges in a centralized architecture is efficiently managing the vast volume of data circulation history—especially when deletion requests require frequent reconstruction of a data's provenance path. While storing full circulation records in SQL-based databases is straightforward, it leads to high storage and query overhead, particularly as data volumes grow.

Our design goal is to ensure the completeness and efficiency of deletion notification distribution, without overwhelming the regulatory infrastructure. Through analysis, we observe that only a small subset of metadata is essential for reconstructing the data-sharing

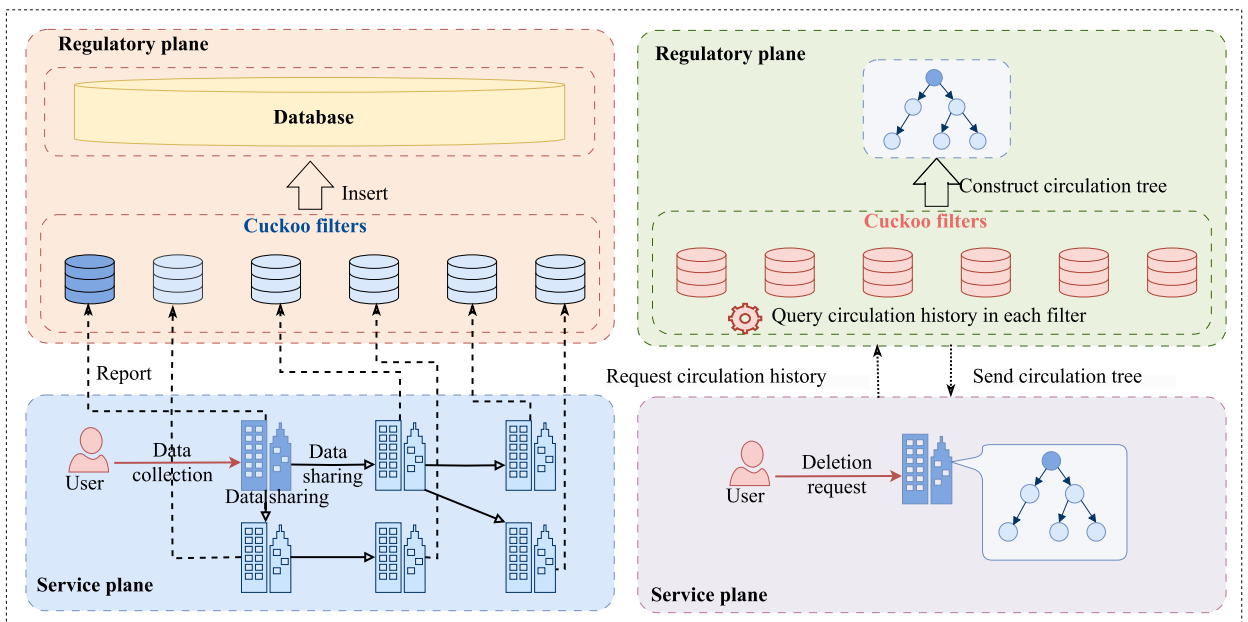


Fig. 4. Cuckoo filters based data circulation recording.

tree: the user ID and data ID. Based on this observation, we propose a solution that encodes circulation edges using cuckoo filters—compact, probabilistic data structures known for fast querying and low memory consumption (as shown in Fig. 4).

At the regulatory plane, we maintain a dedicated cuckoo filter for each registered data controller. When a data-sharing event occurs, for example controller A shares data with controller B, A reports this to the regulator, who then updates B's cuckoo filter as follows: A base fingerprint is computed as Hash of user ID and data's ID. A composite fingerprint is constructed by concatenating the base fingerprint and the ID of the data sender (previous-hop controller). This composite fingerprint is inserted into B's filter, encoding the relationship "A → B" without requiring full SQL queries.

To construct the deletion notification tree, the regulatory plane computes the base fingerprint from a user's deletion request (containing user ID and data ID) and scans all controller filters for matching composite fingerprints. The extracted previous-hop controller IDs form the edges of the data-sharing tree, which is reconstructed using standard spanning-tree algorithms. The detailed implementation is described in Section 5.1.1.

Unlike traditional designs, our approach stores only the previous-hop data controller. Since each transmission has exactly one sender, this representation remains compact and unambiguous. To support full lifecycle management, each cuckoo filter supports three fundamental operations: insertion of data circulation records, deletion tree reconstruction, and deletion of outdated or revoked entries.

4.3. Hop-by-hop verification protocol

Beyond concerns of completeness, the distribution of deletion notifications traversing untrusted networks is susceptible to a spectrum of security vulnerabilities such as tampering, dropping and replay attacks. These vulnerabilities jeopardize the fidelity of the deletion notification process, potentially deviating from the expected procedural integrity. The source data controller necessitates verification that the deletion notification dissemination aligns with the predefined circulation tree. Consequently, there is an imperative to design a deletion right guard within the service plane, one that not only ensures deletion notifications are propagated as intended but also furnishes the initiating data controller with visibility into the outcomes of the notification distribution.

We divide the distribution and acknowledgement of deletion notifications into two distinct subprocesses: the forward distribution of deletion notifications and the backward aggregation of notification acknowledgements (see Fig. 5). The former process involves the sequential dissemination of deletion notifications from the initiating data controller to the leaf nodes within the deletion notification tree, thereby informing the respective data controllers to undertake pertinent deletion actions. Conversely, the latter process entails each leaf node sequentially affirming the receipt of the deletion notification back towards the initiating data controller along the notification tree, facilitating the verification of notification receipt by each data controller.

Given the inherent randomness and unpredictability associated with the history of data circulation, coupled with the aim to minimize system overhead, we introduce a hop-by-hop verification protocol for the forward distribution of deletion notifications. Leveraging message authentication codes, the proposed protocol ensures the accurate delivery of deletion notifications between successive data controllers and adapts to arbitrary data circulation scenarios. For the backward aggregation of notification

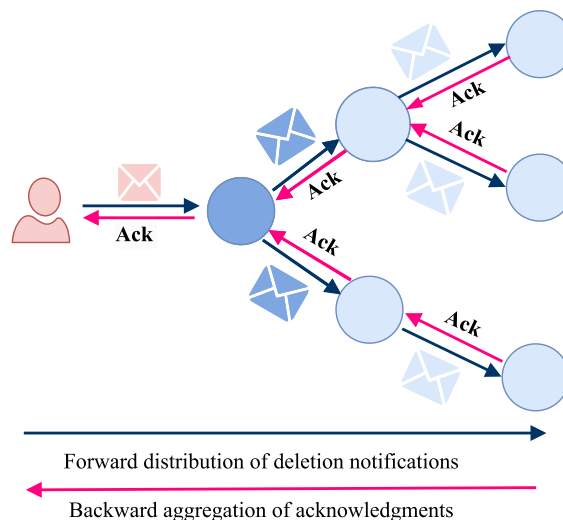


Fig. 5. Hop-by-hop verification.

acknowledgement, we propose a novel traffic-light-inspired recursive tracing mechanism. This approach designates nodes on deletion notification tree into a “red light” state, upon sending out deletion notification and awaiting acknowledgement. Upon receiving a subset of acknowledgements, nodes transition to a “yellow light” state, and ultimately shift to “green light” state once all acknowledgements are received. At the end, the initiating node receives the aggregated acknowledgement to determine if all the involved nodes are correctly notified (see Section 5.2.2 for details).

4.4. TEE-assisted deletion notification distribution network function

To provide an out-of-the-box solution for data controllers, we implement the protocol described in Section 4.3 as a network function. This network function does not affect the normal operations of data controllers. It primarily distributes deletion notifications among data controllers and informs the systems responsible for actual data deletion within the data controllers, to facilitate deletion operations.

We leverage trusted execution environment (TEE) technology, such as Intel's SGX and ARM's TrustZone, to establish a secure enclave within the host that executes the network function for the secure distribution of deletion notifications. Critical security-related operations are confined to the enclave, while other tasks associated with the deletion notification process, such as message body parsing and next-hop data controller determination, operate in the standard memory space. This architecture not only enhances the security of the proposed protocol but also minimizes the overhead associated with transitioning between normal and enclave execution states. Furthermore, through the trusted execution environment, all data controllers involved in the deletion notification distribution process can use the same symmetric encryption key and perform related cryptographic operations in a secure and trusted environment (see Section 5.2.4 for details).

4.5. Adaptive probability sampling for verification

We develop an adaptive probability sampling verification method that achieves accurate violation detection while maintaining low system overhead. For a given time interval, it verifies delivered deletion notification with a high sampling probability when receiving low intensity of deletion requests from clients, while verifying with relatively low probability if large amount of clients enforce their right to deletion.

This approach allows for adaptive risk management by enabling systems to adjust the sampling probability based on the perceived deletion notification intensity. For instance, in scenarios where few deletion notifications are received, the sampling probability should be increased to enhance security measures, since a high sampling rate ensures that malicious attack can be detected and reacted in a timely way. Conversely, when observing a high intensity level of deletion notification, the system can reduce the sampling rate to economize on resources, thereby optimizing the balance between security and operational efficiency (see Section 5.2.3 for details).

5. Proposed mechanism

This section presents the detailed implementation of our proposed solution at both the regulatory and service planes.

5.1. Cuckoo filters based data circulation recording at regulatory plane

As introduced in Section 4.2, we adopt cuckoo filters to compactly record essential data-sharing events. Unlike standard cuckoo filters that only verify existence, our design leverages composite fingerprints to infer parent-child relationships in the data flow. As data volume increases, the filter scales adaptively while preserving accurate and efficient data circulation history recording. Fig. 6 illustrates the mechanism, which operates in two distinct phases: the data-sharing phase and the data-deletion phase.

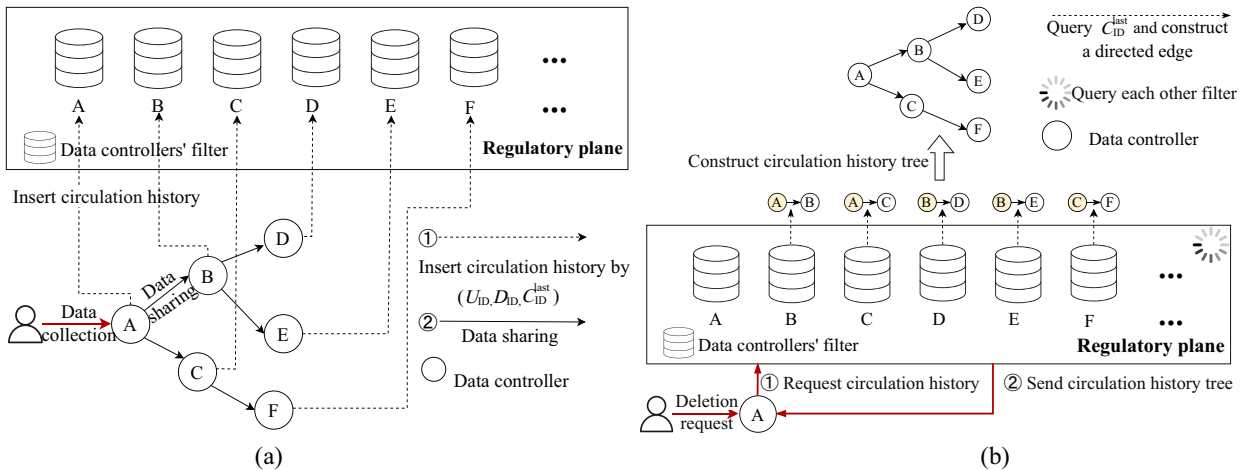


Fig. 6. Workflow of data circulation record insertion and data circulation tree reconstruction. (a) Data sharing phase. (b) Data deletion phase.

5.1.1. Operation during data-sharing phase

The regulatory agency maintains a dedicated cuckoo filter for each registered data controller to store data-sharing history records. When personal data is shared from one data controller to another, the sender must report this event to the regulatory agency. The agency then inserts the corresponding metadata into the filter associated with the receiving controller. Each record is defined by the following key elements: the data ID (D_{ID}), the user ID (U_{ID}), and the ID of the previous-hop data controller (C_{ID}^{last}). To accommodate a growing number of data-sharing records, a dynamic scaling mechanism is integrated into the filter design.

Insertion of data circulation records: As described in Algorithm 1, each insertion uses two types of fingerprints: The base fingerprint (baseFp), derived from D_{ID} and U_{ID} , determines the storage index within the cuckoo filter. The composite fingerprint (compositeFp), which appends C_{ID}^{last} to baseFp, encodes the sharing relationship with the previous data controller. This design allows retrieval of the composite fingerprint based on the base fingerprint, thereby enabling reconstruction of the data circulation tree. The base fingerprint is computed using the MetroHash algorithm, as described by the following equation:

$$\text{hash} = \text{metro.Hash64}(U_{ID} \| D_{ID}). \quad (1)$$

In this formula, metro.Hash64 is a Hash function that generates a 64-bit hash value from the input. Subsequently, the base fingerprint is obtained by performing a modulo 255 operation on the hash value, followed by adding 1 to ensure that the result falls within the range [1, 255]. This procedure utilizes the least significant 8 bits of the hash value, thereby ensuring the fingerprint remains compact while maintaining a uniform distribution. The formula is as follows:

$$\text{baseFp} = (\text{hash} \bmod 255) + 1. \quad (2)$$

The composite fingerprint is generated as:

$$\text{compositeFp} = \text{baseFp} \| C_{ID}^{last}. \quad (3)$$

Storing the composite fingerprint rather than the previous-hop ID alone ensures uniqueness within a bucket, preventing ambiguity during lookups and supporting precise path reconstruction.

Algorithm 1: Insertion of data circulation record

Require: $U_{ID}, D_{ID}, C_{ID}^{last}$.

Ensure: Boolean.

```

1: procedure Insert ( $U_{ID}, D_{ID}, C_{ID}^{last}$ )
2:   Compute hash  $\leftarrow$  metro.Hash64( $U_{ID} \| D_{ID}$ ).
3:   Compute baseFp  $\leftarrow$  (hash mod 255) + 1.
4:   Compute compositeFp  $\leftarrow$  baseFp  $\|$   $C_{ID}^{last}$ .
5:   Compute  $i_1 \leftarrow$  (hash  $\gg$  32) & (bucketsize - 1).
6:   if insertion of compositeFp into position  $i_1$  succeeds then
7:     return true
8:   end if
9:   Compute  $i_2 \leftarrow$  metro.Hash64(baseFp) & (bucketsize - 1).
10:  if insertion of compositeFp into position  $i_2$  succeeds then
11:    return true
12:  end if
13:  Randomly select a position  $i$  from  $\{i_1, i_2\}$ .
14:  for each swap attempt do
15:    Evict the existing fingerprint at  $i$ , insert compositeFp into  $i$ .
16:    Compute the alternative position for the evicted fingerprint.
17:    if maximum swap limit is reached then
18:      return false
19:    end if
20:  end for
21: end procedure

```

During the insertion process, the storage location is first computed based on the base fingerprint (baseFp). The cuckoo filter employs a dual-Hash mechanism, where each composite fingerprint (compositeFp) is assigned two potential storage locations. The initial storage position, i_1 , is calculated as follows:

$$i_1 = (\text{hash} \gg 32) \& (\text{bucketsize} - 1). \quad (4)$$

In this equation, hash $\gg$ 32 extracts the most significant 32 bits of the hash value, leveraging these higher-order bits to mitigate collision occurrences and promote a more uniform distribution of indices. The bitwise AND operation & (bucketsize - 1) is used to ensure the storage location is within the limits of the cuckoo filter, where bucketsize represents the maximum number of fingerprints that can be stored in a single bucket.

In case of the failure of the first insertion attempt, a second storage position, i_2 , is computed, and insertion is attempted again:

$$i_2 = (i_1 \oplus \text{metro.Hash64}(\text{baseFp})) \& (\text{bucketsize} - 1). \quad (5)$$

If both positions are occupied, the cuckoo filter randomly selects one position between i_1 and i_2 , evicts the existing composite fingerprint to its alternative storage location, and inserts the current fingerprint. This process may trigger potential cascaded reinsertion until success or a swap limit is reached.

Dynamic scaling: To effectively accommodate the growing volume of circulation history records, we propose a dynamic scaling mechanism, ensuring that the storage system maintains high-efficiency storage and query performance. When the filter's load factor (i.e., the ratio of occupied storage to total capacity) reaches a predefined threshold, the system automatically expands the storage capacity, typically by doubling the existing capacity. This scaling ensures data integrity and query efficiency, enabling the regulatory agency to rapidly retrieve and delete records. The new storage capacity after scaling is calculated as:

$$\text{NewCapacity} = \text{Currentsize} \times \text{bucketsize} \times 2, \quad (6)$$

where Currentsize represents the current data volume, bucketsize is the capacity of each bucket, and 2 is the scaling factor. The newly allocated storage are appended to the existing filter chain. This not only increases storage but also rebalances distribution to prevent index concentration. The expanded filter supports continued high-efficiency insertion, lookup, and deletion, ensuring system stability and performance in large-scale deployments.

5.1.2. Operation during data deletion phase

Upon receiving a user-initiated deletion request, the regulatory agency must reconstruct a data circulation tree rooted at the data controller receiving the deletion request from user (i.e. source data controller), which helps data controller determine the full scope of deletion notification distribution. The target data circulation tree can be generated by information stored in the cuckoo filters. The system supports two key operations at this phase: querying to retrieve the circulation path, and deletion to remove the associated records, ensuring they are not matched in subsequent queries.

Reconstruction of data circulation tree: As described in Algorithm 2, the agency iteratively queries each cuckoo filter using a base fingerprint derived from the user ID (U_{ID}) and data ID (D_{ID}), which are parsed from the deletion request initiated by the source data controller. For each data controller's filter denoted by C_{ID} , two candidate bucket positions are checked for composite fingerprints whose prefix matches the base fingerprint. When a match is found, a data circulation relationship is identified between the previous-hop controller (C_{ID}^{last}) and the queried data controller C_{ID} . Corresponding edges of the form $(C_{ID}^{\text{last}}, C_{ID})$ are extracted. By aggregating these edges with spanning tree generation algorithm, a data circulation tree rooted at the source data controller can be reconstructed. This reconstructed tree is then sent to the source data controller, enabling it to begin cross-domain deletion notification distribution according to the actual data-sharing history.

Algorithm 2: Reconstruction of data circulation tree

Require: U_{ID}, D_{ID} .

Ensure: Data circulation tree $\mathcal{T}$ or false.

1: **procedure** Tree reconstruction (U_{ID}, D_{ID})

2: Compute hash $\leftarrow \text{metro.Hash64}(U_{ID} \| D_{ID})$.

3: Compute baseFp $\leftarrow (\text{hash} \bmod 255) + 1$.

4: Initialize an empty list of edges $edges$.

5: **for** each cuckoo filter cf maintained by regulatory plane **do**

6: Compute $i_1 \leftarrow (\text{hash} \gg 32) \& (\text{bucketsize} - 1)$.

7: Compute $i_2 \leftarrow \text{metro.Hash64}(\text{baseFp}) \& (\text{bucketsize} - 1)$.

8: **for** each position i in $\{i_1, i_2\}$ **do**

9: **for** each fingerprint fp in $cf.buckets[i]$ **do**

10: **if** fp has prefix matching baseFp **then**

11: Extract $C_{ID}^{\text{last}} \leftarrow$ suffix of fp after baseFp.

12: Add edge $(C_{ID}^{\text{last}}, cf.ID)$ to $edges$.

13: **end if**

14: **end for**

15: **end for**

16: **end for**

17: **if** $edges$ is not empty **then**

18: Construct the data circulation tree $\mathcal{T}$ from $edges$.

19: **return** $\mathcal{T}$

20: **else**

21: **return** false

▷No circulation tree found

22: **end if**

23: **end procedure**

Deletion of data circulation records related to data be deleted: Once the regulatory agency verifies that all data controllers in the data circulation tree have successfully deleted the target data, it proceeds to purge the corresponding circulation records from the cuckoo filters. This deletion process leverages a key advantage of cuckoo filters: support for efficient item removal. This is one of the reasons we adopt this probabilistic data structure.

To execute this cleanup, the regulatory plane can adopt one of two strategies depending on system requirements and operational constraints. The first approach is a proactive strategy: During the construction of the data circulation tree, the agency pre-records the indices of relevant composite fingerprints. Once the deletion is confirmed, these pre-identified entries are immediately removed from the associated filters, minimizing overhead at deletion time. The second approach is a reactive strategy: To conserve memory and reduce temporary state storage, the agency reconstructs the data circulation tree at the moment deletion is triggered (refer to Algorithm 2). It then proceeds to locate and remove matching composite fingerprints across all controller filters.

Both strategies are functionally equivalent in terms of final outcome. The choice between them is delegated to the regulatory authority, which can make an informed decision based on current storage availability, system load, or policy requirements. In either case, the removal of outdated circulation records ensures that deleted data cannot be traced, queried, or leaked in the future—safeguarding compliance and preserving individual privacy.

5.2. TEE-assisted hop-by-hop verification at service plane

The service plane, consisting of multiple data controllers, is responsible for orchestrating the deletion process of personal data. As data circulates among data controllers, each controller reports its circulation records to the regulatory agency for storage. When a user submits a deletion request, the service plane requests the data circulation tree from the regulatory agency. This tree visually represents the data circulation path and is utilized by the source data controller to forward deletion notifications to its directly connected next-hop data controller, requesting the deletion of the specified data. Each recipient data controller continues forwarding the deletion instruction to its subsequent hop, and this process repeats until all involved data controllers complete the deletion. After verifying the deletion notification successfully, each data controller returns a deletion confirmation to its previous hop.

5.2.1. Forward traversal verification

DelRightGuard protocol defines a standardized process for handling and forwarding deletion notifications, ensuring that all relevant data controllers receive the deletion notification. Through the protocol header fields, the DelRightGuard protocol allows the source, destination, and necessary operation instructions of the deletion notification to be clearly identified and conveyed, thereby achieving precise control and supervision of data deletion notifications. The core protocol header's design includes the user's unique identifier (U_{ID}), notification type (Type), timestamp (T), the target data controller's ID (C_{ID}^{des}), operation notification (Req), the data circulation tree (Tree), and authentication field (Auth) (refer to Fig. 7).

The header fields of the protocol are designed as follows:

- U_{ID} represents the unique identifier of the user in the deletion notification.
- Type is used to distinguish the source of the deletion notification, with Type = 0 indicating issued by the user, Type = 1 indicating the command is forwarded by the data controller, and Type = 2 representing a deletion notification acknowledgement.
- C_{ID}^{des} is identifier of the next-hop data controller receiving the deletion notification. This field is not used when Type = 2.
- T represents the timestamp, used to compute $\Delta t = t_2 - t_1$ for freshness verification, preventing replay attacks by ensuring notifications are not outdated.
- Req encodes specific deletion operations (e.g., erasing all personal information, age data, or identification numbers), with mappings and extensions customizable by data controllers based on their operational standards.
- Tree represents the dynamic history of data propagation. Initially, it includes all controllers responsible for the deletion process. As notifications are forwarded, each controller eliminates previous nodes, retaining only its own identifier and those of the subsequent controllers.

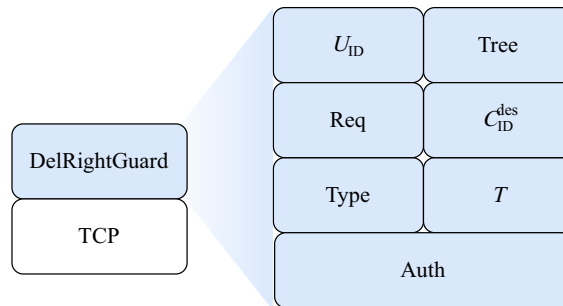


Fig. 7. Protocol header structure of DelRightGuard.

- Auth is constituted by a quadruple and session key k , as follows:

$$\text{Auth} \leftarrow \text{HMAC}((U_{\text{ID}} \parallel \text{Type} \parallel T \parallel C_{\text{ID}}^{\text{des}} \parallel \text{Req} \parallel \text{Tree}), k). \quad (7)$$

In the DelRightGuard protocol, hop-by-hop verification (refer to section 4.3) is used to ensure the secure transmission of deletion notifications. Whenever a deletion notification is forwarded to the next data controller, the receiving data controller replaces the target data controller ID ($C_{\text{ID}}^{\text{des}}$) in the protocol header with its own ID, and then recalculates the authentication field Auth' based on the updated information:

$$\text{Auth}' \leftarrow \text{HMAC}((U_{\text{ID}} \parallel \text{Type} \parallel T \parallel C_{\text{ID}}^{\text{own}} \parallel \text{Req} \parallel \text{Tree}), k). \quad (8)$$

In this way, the authentication at each hop is based on the unique information of the current recipient, enhancing the security of the message transmission process. Through this hop-by-hop authentication method, the DelRightGuard protocol can effectively prevent data tampering and forgery, ensuring that deletion notifications are transmitted securely and reliably among the data controllers.

5.2.2. Recursive backward acknowledgement

We propose an innovative traffic light mechanism (TLM) for managing the acknowledgement process of deletion notifications. TLM controls and guides the transmission of acknowledgement messages within the data controller through three main phases: the red light phase, the yellow light phase, and the green light phase, refer to Fig. 8 for the schematic diagram and Algorithm 3 for the algorithm.

Algorithm 3: TLM algorithm

Input: Root node of a data controller network.
Output: Network processed following TLM.

```

1: procedure TLM (root)
2:    $Q \leftarrow$  empty queue.
3:    $R \leftarrow$  empty list.
4:   Enqueue (root, 'INIT') into  $Q$ .
5:   while not  $Q.\text{isEmpty}(\cdot)$  do
6:     (node, phase)  $\leftarrow Q.\text{dequeue}(\cdot)$ .
7:     if phase = 'INIT' then
8:       Enqueue (node, 'RED') into  $Q$ .
9:     else if phase = 'RED' then
10:      if security check passes then
11:        Enqueue (node, 'YELLOW') into  $Q$ .
12:      else
13:        Enqueue (node, 'ALERT') and its ancestors into  $Q$ .
14:      end if
15:    else if phase = 'YELLOW' then
16:      if all children confirmed or no children then
17:        Enqueue (node, 'GREEN') into  $Q$ .
18:         $R.\text{append}(\text{node})$ .
19:      else
20:        Wait for all children to confirm.
21:         $R.\text{extend}(\text{children\_acknowledgements})$ .
22:      end if
23:    else if phase = 'GREEN' then
24:      Notify parent, if any, and adjust parent's state accordingly, transmit  $R$  to parent.
25:    else if phase = 'ALERT' then
26:      Process security alert, notify all relevant parties.
27:    end if
28:  end while
29: end procedure

```

- **Red light phase:** In the red light phase, each data controller, upon receiving a deletion notification, first conducts a security verification to check if the notification has been compromised (e.g., tampered with or forged). If the verification fails, indicating that the deletion notification has been attacked, the data controller immediately reports this to its superior data controller, enters an ALERT state, and relays the attack warning up the hierarchy to the source data controller, thereby halting the forwarding of the deletion notification. If the verification is successful, the data controller uses a counter to track the number of child nodes awaiting acknowledgement, preparing to move to the next phase.
- **Yellow light phase:** After the notification has passed the security verification, it enters the yellow light phase, where the data controller awaits acknowledgement from all subordinate nodes. Once the data controller receives acknowledgement from all its subordinate nodes, it will enter the green light phase.
- **Green light phase:** In the green light phase, the data controller, upon confirming that it has received acknowledgement from all its subordinate nodes, sends an acknowledgement message back to its superior data controller node. This state indicates that the data controller and all its subordinate nodes have successfully processed the deletion notification, and the entire branch is ready to proceed with normal operations.

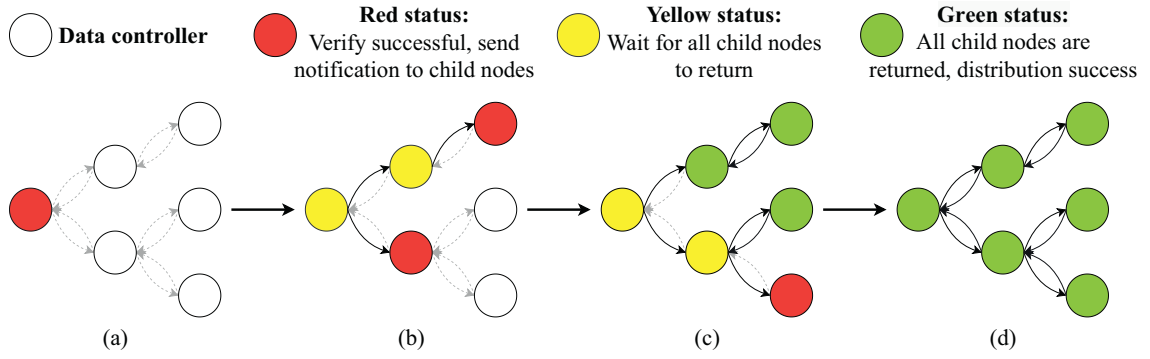


Fig. 8. TLM schematic diagram.

5.2.3. Probabilistic sampling verification

To reduce the communication cost of data controllers during tag verification, we propose a probabilistic sampling verification method. This mechanism dynamically adjusts the sampling probability based on the number of deletion notifications processed by the data controller, effectively reducing the operational burden on data controllers. In the specific operation process, each data controller first performs a keyed pseudorandom function PRF on the Auth field using the session key k , and then compares the result with a predefined sampling probability:

$$\text{PRF}_k(\text{Auth}) \leq P_s. \quad (9)$$

If the above conditions are met, the data controller will verify Auth and send an acknowledgement notification. Moreover, each data controller maintains a counter, CountS, to record the number of deletion notifications processed. The sampling probability P_s is dynamically calculated through CountS and a predefined constant u . As CountS increases, the sampling probability correspondingly decreases; when CountS is small, the sampling probability increases, allowing for the verification of more notifications. The calculation method is as follows:

$$P_s(\text{CountS}) = \frac{1}{(1+u)^{\text{CountS}}}. \quad (10)$$

However, directly applying this sampling formula could result in a too low sampling probability after processing a large number of deletion notifications. To address this issue, we introduce a minimum sampling probability, $P_{\min}$, into the sampling formula to ensure that the sampling probability does not fall too low. Different data controllers should adjust the relevant parameters according to their own application scenarios and requirements to achieve the best efficiency and cost balance. Therefore, the final calculation formula for the sampling probability is as follows:

$$P_s(\text{CountS}) = \begin{cases} \frac{1}{(1+u)^{\text{CountS}}}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} > P_{\min} \\ P_{\min}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} \leq P_{\min} \end{cases}. \quad (11)$$

Specific parameter configurations should be flexibly adjusted by data controllers based on actual needs. When higher security requirements are needed, the sampling rate can be appropriately increased to enhance the system's security performance. Additionally, data controllers can also decide on the most suitable parameter settings based on various factors such as data volume, processing speed, and system load.

5.2.4. Secure NF execution

Our trust distribution verification mechanism is implemented on Intel SGX (refer to Fig. 9 for more details). As one of the leading trusted execution environment (TEE) technologies, SGX creates a secure container called an enclave through extended instruction sets and memory access control. This container carves out a protected area within the application's address space, aimed at providing sealed confidentiality and integrity protection for code and data within the container. In our scheme, the verification process of commands will take place within the enclave, and the keys required for verification will also be securely stored therein. Thanks to the isolation design between the enclave and the data controller system, attackers or dishonest data controllers cannot access the code or keys within the enclave (the latter being encrypted in memory), nor can they tamper with or falsify verification results. This greatly enhances the supervisory ability over the deletion notification circulation process, ensuring the security and reliability of the verification mechanism.

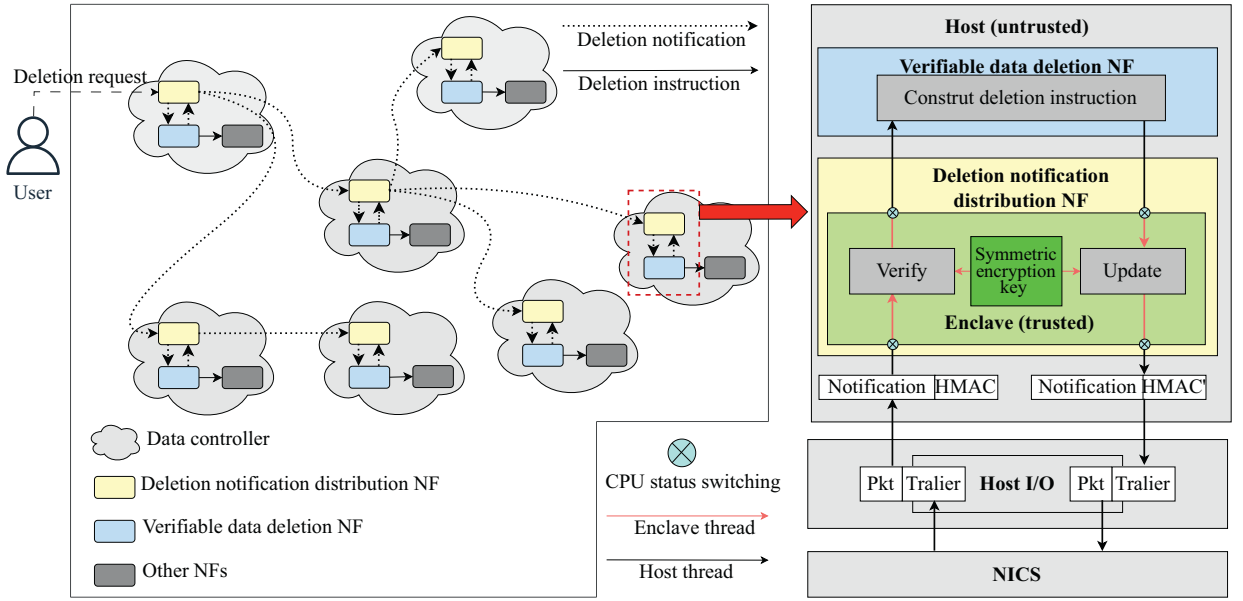


Fig. 9. Secure NF execution.

6. Security analysis

In this section, we analyze the robustness of DelRightGuard against the attacks outlined in the threat model (Section 3.2), focusing on tampering, impersonation, path deviation, replay, and drop attacks.

6.1. Tampering of deletion notification

In this attack, Eve intercepts the deletion notification and modifies Alice's ID to Bob's ID before forwarding it to the target data controller. Our protocol uses hop-by-hop verification, where each data controller along the communication path verifies the message using message authentication codes. If a notification is tampered with, the Auth will fail to validate, and the message will be rejected. Additionally, all session keys used for authentication are securely stored within the trusted execution environment (TEE), ensuring that even in compromised systems, attackers cannot gain access to these keys. The TEE protects cryptographic operations, making it virtually impossible for an attacker to alter the notification without being detected. Periodic key rotation further enhances long-term protection.

6.2. Impersonation attack

The malicious attacker, Eve, having obtained Alice's ID, attempts to masquerade as a node in Alice's personal information flow tree and sends a false deletion notification to the target data controller that holds Alice's private information. The hop-by-hop verification mechanism ensures that each controller verifies the authenticity of its predecessor using session keys stored securely in the TEE. As these keys are inaccessible outside the TEE, attackers cannot generate valid Auth, rendering impersonation attempts futile. Furthermore, the traffic light mechanism (TLM) provides an additional layer of verification by requiring each controller to send acknowledgements back through the notification chain. This recursive backward mechanism ensures that all controllers involved are properly authenticated and prevents impersonation attempts from succeeding.

6.3. Path deviation attack

The deletion notification issued by Alice is intercepted by the malicious attacker, Eve, who alters the destination address and sends it to a non-target data controller. Our hop-by-hop verification protocol effectively prevents such deviations by requiring each controller to verify the sender's identity before forwarding the notification. If an unauthorized controller attempts to alter the path, the verification will fail and the notification will be rejected. Moreover, the recursive TLM ensures that every controller in the notification chain receives and acknowledges the notification. This dual-layer approach guarantees that the deletion process follows the correct path and maintains its integrity.

6.4. Replay attack

The deletion notification forwarded between data controllers is intercepted and saved by the malicious attacker, Eve, without any modifications. After a certain period, it is resent to the target data controller. To mitigate this, each notification includes a timestamp.

The target data controller successfully identifies the replay attack through the detection of the timestamp Δt . The TLM mechanism further strengthens defense against replay attacks, as acknowledgements are recursively sent back to the source controller, allowing the system to detect and reject repeated notifications based on outdated timestamps.

6.5. Drop attack

Data controller M forwards Alice's deletion notification to data controller N, but the malicious attacker Eve intercepts and discards it, then sends a false deletion success acknowledgement to data controller M. Data controller M conducts verification on the response message and finds a mismatch, indicating an anomaly in the sending of the deletion notification. Our protocol uses the TLM to ensure that acknowledgements are sent back recursively through the notification chain. If any controller fails to send an acknowledgement, the upstream controllers will detect the missing confirmation and flag the deletion process as incomplete. This prevents attackers from discarding notifications without detection.

7. Performance evaluation

7.1. Implementation and experimental setup

We design and develop a prototype system for the proposed solution. On the regulatory plane, we utilize the Go language to implement a data circulation tree query mechanism based on cuckoo filters. On the service plane, we implement the proposed secure distribution protocol for deletion notifications, with Python Sanic framework², employing asynchronous I/O technology and RESTful APIs.

Our experiments are conducted using containernet, a network emulation tool based on mininet, which is enhanced by Docker integration³. In our emulated network environment, each Docker container functions as a data controller. To simulate the deletion notification distribution process and accurately evaluate the performance of our proposal in real-world network infrastructures, we employ 4 notable network topologies, Chinanet, Karen, Lambdanet, and Viatel, from the Internet Topology Zoo [35]. These network topologies in Topology Zoo is described in GraphML format. Each GraphML file encompasses diverse network nodes and interconnections. Containernet-based network emulation is started by parsing GraphML into docker container and virtual network links.

The emulation program is executed on a virtual machine (VM) hosted by Aliyun Cloud. The VM's configuration for the study is as follows: an ecs.g7t.16xlarge cloud server, supporting Intel SGX for trusted execution environments, and equipped with 128 GB of encrypted memory. Additionally, the server is provisioned with 64 vCPUs, 256 GB of memory, and 32 GiB of network bandwidth.

7.2. Performance evaluation

Given that no existing solutions have been proposed in the literature for the distribution of deletion notifications across multiple data controllers, we are unable to conduct a direct longitudinal comparison with other specific solutions. However, since TLS is a commonly considered approach for secure communication, we have opted to compare DelRightGuard against TLS to highlight the differences. We focus on both the regulatory plane and the service plane, with an emphasis on the computational overhead and scalability.

7.2.1. Regulatory plane

Setup: We conducted a series of experiments to evaluate the performance of our cuckoo filter-based mechanism for recording data circulation history, as described in Section 5.1. Our evaluation focuses on average operation time, measured across the core operations of insertion, lookup, and deletion. These metrics reflect the system's efficiency and scalability under varying loads.

The experiments assess the impact of two key parameters. The first is default capacity, which defines the initial number of records and each filter can accommodate before triggering dynamic scaling. The other is bucket size, the number of fingerprints per bucket, which affects collision handling and lookup cost. We test combinations of default capacities (10,000 and 100,000) and bucket sizes (8, 16, and 32) to simulate both lightweight and large-scale deployments.

Performance evaluation: Experimental results are presented in Fig. 10. The x-axis denotes the number of data circulation records store of each cuckoo filter (displayed on a logarithmic scale). The y-axes of Fig. 10(a), (b), and (c) correspond to the average times per circulation history insertion, circulation history search, and circulation history deletion.

As shown in Fig. 10(a), the average insertion time remains below 1 μ s, even for 300 million circulation records. This demonstrates that our method efficiently handles large-scale data ingestion. With the same bucket size, filters with higher default capacity exhibit slightly longer insertion times due to increased hash conflict resolution. While larger bucket sizes reduce insertion failures, they also increase traversal overhead, resulting in marginally higher latency.

From Fig. 10(a), we observe that the average insertion time slightly increases as the data circulation records grow. Even with 300 million circulation histories, the peak remains below 1 μ s. This demonstrates our method efficiently handles large-scale data insertions of circulation histories. Next, we find that with the same bucket size, higher capacities result in a smaller increase in time. Because higher capacities require less frequent dynamic scaling, thereby reducing the time needed. At last, We find that as bucket size increases,

² <https://github.com/sanic-org/sanic>.

³ <https://github.com/containernet>.

insertion time also increases. This is because, in cuckoo hashing, each element is assigned two potential positions. If both positions are occupied, the element must be relocated to make space. Large bucket sizes can decrease conflicts. However, when conflicts occur, they trigger chain reactions, increasing the resolution time.

Next, we analyze Fig. 10(b). We notice that as the number of circulation history increases, time also increases. This trend is expected, as more circulation history in the search process requires traversing more positions. Next, with the same bucket size, we find that higher capacities result in shorter search times. This is because a larger default capacity has a lower load factor, making the elements more sparsely distributed and reducing the search time in each bucket. Lastly, with a constant default capacity, configurations with bucket sizes of 8 and 16 show better performance, whereas a bucket size of 32 results in longer search times. Because a larger bucket size requires more comparisons during a search within a bucket, this leads to longer search times for data circulation records.

We find that the average deletion time per circulation history (refer to Fig. 10(c)) is similar to the average search time because the deletion operation of data circulation records is based on the search operation.

Based on the above analysis, we can conclude that our proposed centralized management architecture (see Section 4.1) can efficiently handle large-scale data circulation history while maintaining a lightweight operational profile. The experimental results clearly demonstrate the system's ability to process up to 300 million circulation history records with minimal performance degradation. For instance, the average insertion time per record remains below 1 μ s, showcasing the system's high efficiency even under heavy loads.

In summary, choosing a smaller bucket size and a larger initial capacity significantly reduces both operation time and memory footprint. Regulatory agencies could dynamically adjust filter parameters based on the actual number of circulation histories, ensuring optimal operation across various deployment scales.

7.2.2. Service plane

Setup: To assess the resource efficiency of our lightweight solution implemented in the service plane, we conducted a series of experiments focusing on three key metrics: CPU usage, memory consumption, and throughput. Our objective is to demonstrate that DelRightGuard not only ensures secure deletion notification distribution but also achieves this with minimal resource overhead and high processing efficiency, highlighting its lightweight design.

For comparison, we use a TLS-based scheme, where adjacent data controllers rely on TLS for secure deletion notification delivery—a commonly adopted approach. To evaluate the cost of TEE to enhance the security of NF, we measure resource consumption both within and outside the trusted execution environment (TEE). Additionally, we explore the effect of adaptive probabilistic sampling on system resource utilization. In the adaptive probabilistic sampling mechanism, the sampling probability is dynamically adjusted based on the volume of deletion notifications processed, with an initial sampling probability of 0.7 and a minimum threshold of 0.3 to ensure continuous verification. To strike a balance between security and resource overhead, each data controller should choose appropriate sampling parameters when using adaptive probabilistic sampling, based on the volume of deletion notifications processed and system load.

Finally, to ensure the robustness of our lightweight design across various network conditions, we perform experiments on four representative network topologies: Chinanet, Karen, Lambdanet, and Viatel.

Performance evaluation: The results of the throughput evaluation are presented in Fig. 11. In each subfigure, the x-axis represents the number of deletion notifications received by each data controller per second, signifying the intensity of deletion notifications. Correspondingly, the y-axis illustrates the measured throughput, which is expressed in messages per second (msg/s). This unit reflects the rate at which the system can process messages or deletion notifications. Specifically, it measures how many messages the system can handle and respond to in a given time frame. A higher throughput value on the y-axis indicates that the system is capable of processing more messages per second, demonstrating better scalability and efficiency in handling deletion notifications under varying loads.

Taking the experimental results under the Chinanet topology as an example, particularly the throughput depicted in Fig. 11(a), it is evident that the method based on TLS exhibits the lowest throughput. Specifically, throughput declines as the intensity of deletion notifications increases, decreasing from approximately 24,238.74 messages per second at 100 notifications per second to 16,111.13 messages per second at 5000 notifications per second. This reduction is substantially lower than that observed for other methods. The diminished throughput of the TLS-based approach can be attributed to its reliance on computationally intensive asymmetric encryption

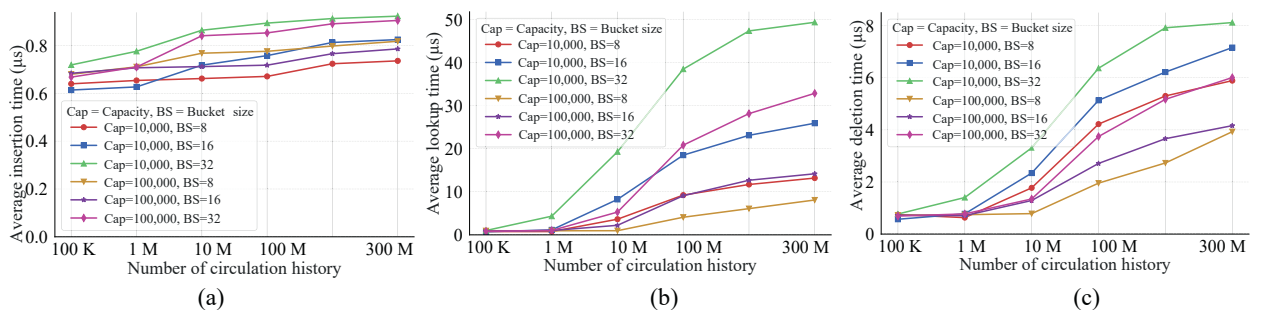


Fig. 10. Performance evaluation on regulatory plane. (a) Insertion time per data circulation record. (b) Lookup time per data circulation record. (c) Deletion time per data circulation record.

and decryption operations. In contrast, DelRightGuard leverages symmetric cryptography, significantly improving throughput relative to the TLS-based method. The observed increase in throughput indicates that, under the proposed solution, a data controller can process a larger volume of deletion notifications more efficiently.

Regarding the impact of SGX utilization on throughput, the findings indicate that the adoption of SGX results in a minimal reduction in throughput. This decline is primarily due to the I/O operations between the enclave and non-enclave memory regions. However, the impact is mitigated by the architectural design, which confines only essential functionalities—specifically, the verification components—within the SGX enclave. This strategic confinement minimizes the computational overhead within the enclave, thereby limiting the adverse effects on throughput.

With respect to throughput variations associated with adaptive probabilistic sampling, a noticeable difference emerges when the number of deletion notifications exceeds 1000 per second. As the intensity of deletion notifications continues to increase, the advantages of employing adaptive probabilistic sampling become increasingly evident, as it substantially enhances throughput compared to scenarios without sampling. This improvement is due to the reduction in the number of deletion notifications requiring full verification, thereby alleviating computational demands. Consequently, data controllers can dynamically adjust sampling configurations to optimize system performance while accommodating the demands of processing deletion notifications.

The experimental results from the other three network topologies (refer to Fig. 11(b)–(d)) exhibit a similar trend to those observed under the Chinanet topology, further confirming the lightweight nature of our solution. Across all network conditions, DelRightGuard consistently maintains high throughput, with adaptive sampling further enhancing performance efficiency.

The results of the CPU and memory footprint evaluation are presented in Fig. 12. In each subfigure, the x-axis denotes the number of deletion notifications received by each data controller per second, indicating the intensity of deletion notifications. Concurrently, the y-axis depicts the corresponding CPU utilization and memory footprint.

Considering the experimental results obtained under the Chinanet topology as an example, particularly in terms of CPU usage illustrated in Fig. 12(a), we observe that the method based on TLS exhibits the highest CPU consumption. The CPU usage escalates with the increasing intensity of deletion notifications, reaching approximately 60% at a rate of 5000 notifications per second, which is notably higher than that of other methods. This increased resource demand can be attributed to the TLS method's reliance on computationally intensive asymmetric encryption and decryption operations. In contrast, DelRightGuard employs symmetric cryptography, which substantially lowers CPU usage compared to the TLS-based approach. A reduction in CPU consumption implies that,

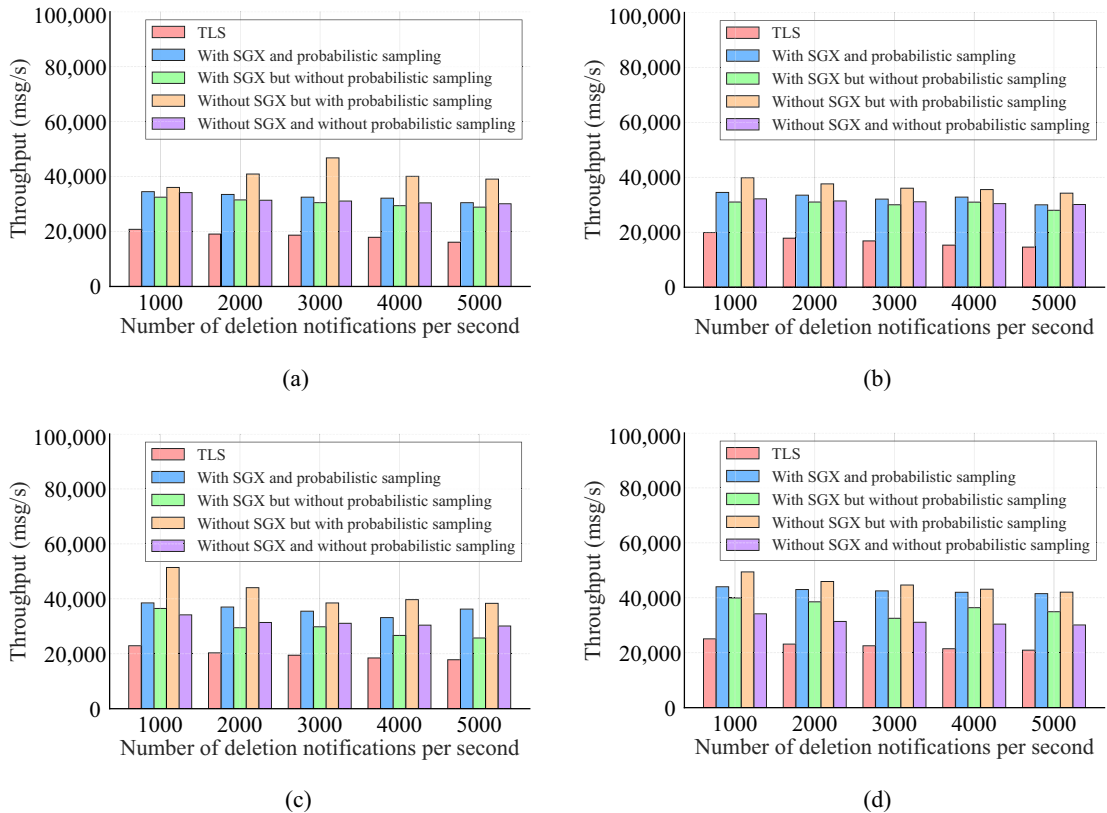


Fig. 11. Throughput with respect to the intensity level of deletion notifications under different network topology. (a) Throughput under Chinanet. (b) Throughput under Karen. (c) Throughput under Lambdanet. (d) Throughput under Viatel.

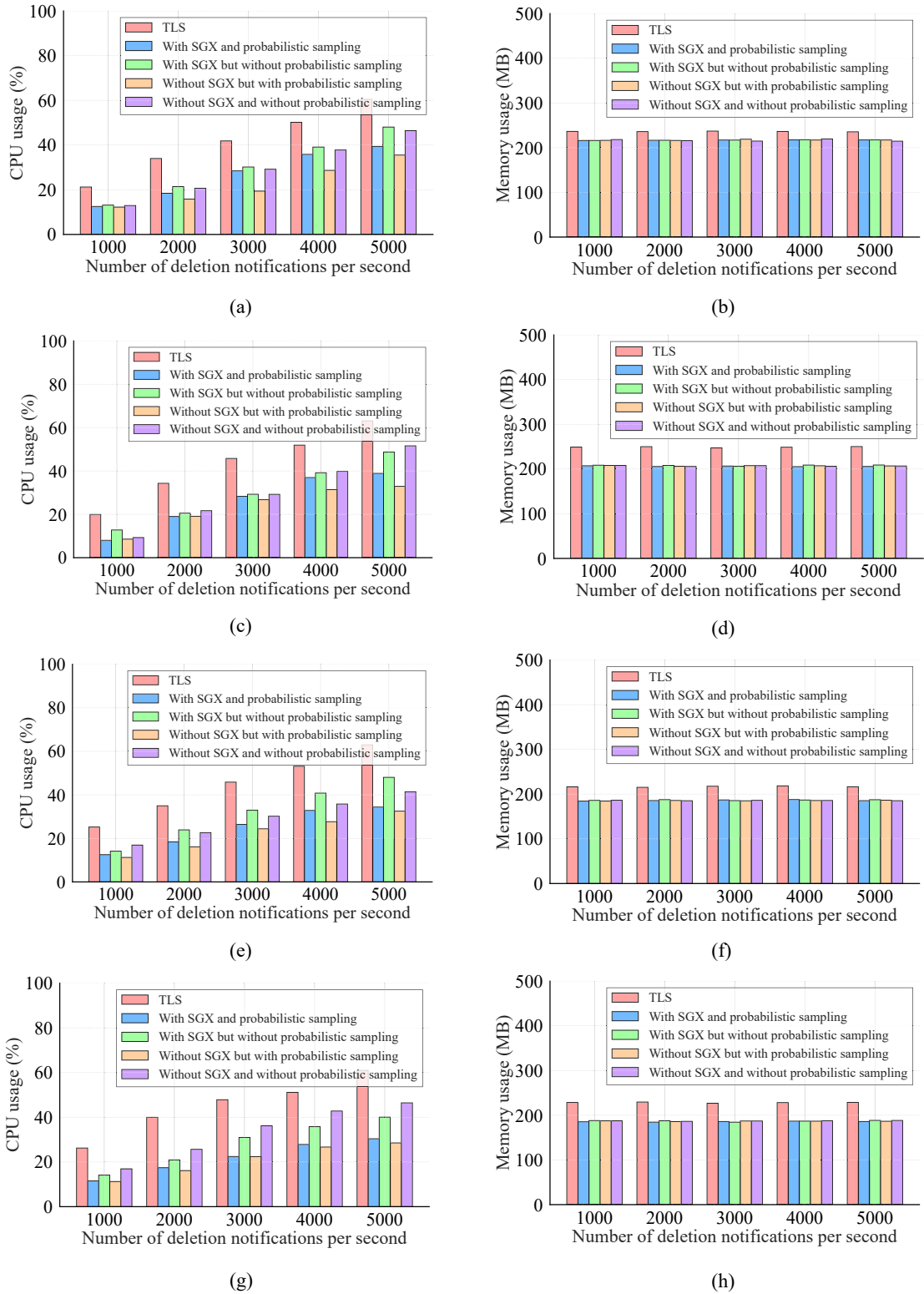


Fig. 12. CPU usage and memory footprint with respect to the intensity level of deletion notifications under different network topology. (a) CPU usage under Chinanet. (b) Memory footprint under Chinanet. (c) CPU usage under Karen. (d) Memory footprint under Karen. (e) CPU usage under Lambdanet. (f) Memory footprint under Lambdanet. (g) CPU usage under Viatel. (h) Memory footprint under Viatel.

under our proposed solution, a data controller is capable of handling a higher volume of deletion notifications more efficiently.

Regarding the impact of utilizing SGX on CPU resource consumption, it is observed that the adoption of SGX leads to a minimal increase in CPU usage, predominantly due to the I/O operations between the enclave and non-enclave areas. This marginal rise in CPU demand is attributed to the design of only the essential functionalities—specifically, the verification components—within the SGX enclave, thereby minimizing the computational burden inside the enclave.

Concerning the CPU resource consumption with and without adaptive probabilistic sampling, a clear difference in CPU usage is noticeable when the number of deletion notifications exceeds 1000 per second. With further increase of deletion notification intensity, the benefit of using sampling becomes very clear, as it significantly lowers CPU usage compared to not using sampling. This decrease in CPU load happens because sampling reduces the amount of deletion notifications the CPU needs to verify. As a result, data controllers have the flexibility to tweak sampling settings based on their specific needs, helping them find the right balance between system performance and the demands of processing data.

In Fig. 12(b), we see that the memory footprint for the TLS solution stays around 118 MB, whereas our solution uses about 107 MB, approximately 10% less than the TLS solution. This difference largely stems from TLS's reliance on asymmetric encryption and decryption algorithms, which consume more memory for key storage and intermediate computations. Interestingly, the use of SGX does not notably change the memory footprint; SGX is designed to enhance security without significantly impacting memory footprint. Similarly, employing a sampling strategy does not affect the memory footprint substantially. Sampling aims to reduce the data volume for processing, which mainly influences CPU workload and processing times rather than memory use. Regardless of the employed method, the overall memory footprint remains relatively stable, attributed to the I/O-intensive nature of the operations, where memory consumption is not heavily dependent on data volume. The significant rise in CPU usage with increasing data volume is linked to the use of asynchronous or non-blocking I/O, which boosts efficiency by allowing the CPU to handle other tasks while awaiting I/O completions. Consequently, as the amount of I/O operations grows, so does CPU utilization, enhancing the overall performance of the application.

The experimental results from the other three network topologies (refer to Fig. 12(c)–(h)) exhibit a similar trend to those observed under the Chinanet topology, further confirming the lightweight nature of our solution. Across all network conditions, DelRightGuard consistently maintains low CPU usage and minimal memory overhead, with adaptive sampling further reducing resource consumption. This demonstrates its efficiency in managing deletion notifications, even in diverse and complex environments. This consistency highlights that our solution operates with minimal resource consumption, regardless of the underlying network topology, affirming the system's scalability and flexibility in real-world deployment scenarios. This reveals that our solution is independent of network topology. This is due to the design of the hop-by-hop verification (detailed in Section 4.3), which decouples the operation of our proposal from network topology. Thanks to this design, our solution can cope with arbitrary network topology and arbitrary data circulation trees.

The experimental results clearly demonstrate that, compared to TLS, DelRightGuard significantly reduces computational overhead, providing a more efficient and scalable solution for secure deletion notification distribution. Its lightweight performance makes DelRightGuard particularly well-suited for large-scale, cross-domain environments where resource efficiency and operation time are critical. Additionally, DelRightGuard is capable of effectively managing various network topologies and data circulation trees, ensuring efficient operation across diverse deployment environments.

Moreover, data controllers can adjust sampling settings to balance performance and data processing demands. The parameter u controls how quickly the sampling probability decreases as more deletion notifications are processed. A higher u reduces the probability faster, ideal for resource-constrained environments, while a lower u ensures stricter security. The minimum sampling probability $P_{\min}$ maintains verification under heavy load, with lower values conserving resources and higher values supporting more frequent verification.

8. Discussion

8.1. Comparison with existing methods

In recent years, with the growing awareness of the right to deletion, research on data deletion protection has expanded significantly. While numerous studies have explored areas such as irrecoverable deletion, verifiable deletion, and machine unlearning, none have adequately addressed the challenge of securely distributing deletion notifications across multiple data controllers in untrusted networks. Existing methods primarily focus on single controllers or internal systems, overlooking the complexities of cross-domain notification. To bridge this gap, our work introduces the DelRightGuard protocol, a lightweight and efficient mechanism designed specifically for secure deletion notification distribution across untrusted networks.

DelRightGuard builds upon HMAC and TEE to address the overlooked issue of secure deletion notifications. This approach is common in network engineering and cybersecurity research, as these fields prioritize processing efficiency and tend to favor lightweight cryptographic methods. The decision to use HMAC and TEE stems from their low overhead, making them suitable for our scenario, particularly in handling large-scale deletion requests. On top of HMAC and TEE, DelRightGuard introduces an innovative combination of hop-by-hop verification, adaptive sampling, and a cross-layer cooperation centralized management architecture. This integrated mechanism ensures the secure and efficient distribution of deletion notifications across multiple data controllers, particularly in large-scale, cross-domain scenarios. Effectively tackle the secure distribution of deletion notifications between data controllers challenge that existing methods fail to consider. Furthermore, compared to TLS, which might be a direct and commonly considered solution for secure communication, DelRightGuard offers significantly lower computational and network overhead, making it more efficient for large-scale, cross-domain notification distribution.

8.2. Applicability in real world

One may challenge that the application scenarios proposed in this work, along with the centralized management architecture, face inherent scalability issues. This is a well-recognized challenge for centralized systems. However, we argue that our approach can effectively address these concerns for several reasons:

- (1) **Unique characteristics of deletion notification:** Unlike high-demand applications like video streaming, which require sustained bandwidth and processing power, the deletion notification scenarios in our system are inherently lightweight. These notifications are processed quickly and are transient, which minimizes the load. The brief and low-bandwidth nature of deletion notifications ensures that the system can handle a large volume of requests without encountering performance bottlenecks, allowing for smooth and efficient operation.
- (2) **Distributed regulatory architecture:** To address scalability and compliance challenges in large-scale networks with numerous data controllers, we propose a future enhancement in the form of a “distributed regulatory architecture”. In this model, regional regulatory centers would be responsible for managing deletion notifications within their specific geographic areas or jurisdictions. By allowing these centers to operate independently, the system effectively distributes both the computational and operational load. This approach reduces the likelihood of single-point failures and enhances overall scalability. With regional centers in place, the system can efficiently handle deletion notifications in even the largest deployments, avoiding excessive strain on any one central node and significantly minimizing the risk of single-point bottlenecks.
- (3) **Leveraging best practices from centralized systems:** Additionally, our approach can draw upon the best practices of existing centralized systems, such as load balancing and hierarchical architectures commonly used in DNS, SDN, and other well-established networks. These systems have successfully managed scalability and reliability in complex environments, and by incorporating similar mechanisms, we can further mitigate potential challenges in scaling our centralized design. By applying these proven strategies, our system can balance scalability with efficiency while minimizing the risk of single-point failures.

9. Conclusion

In an era of widespread personal data sharing, securely and completely distributing deletion notifications across multiple data controllers is essential to uphold the right to deletion. This paper is the first to identify this challenge and propose a comprehensive solution. We introduce DelRightGuard, a cross-plane collaborative protocol that combines a cuckoo filter-based data circulation recording mechanism at the regulatory plane with a secure and lightweight deletion notification distribution protocol at the service plane. Security analysis confirms its robustness against potential attacks, validating its suitability for real-world deployment in privacy-compliant ecosystems. Experimental results show that the system maintains microsecond-level performance and scales to hundreds of millions of data circulation records. On the service plane, performance evaluations confirm that the proposed deletion notification distribution protocol securely manages large volumes of deletion requests in a lightweight and efficient manner.

Future work will explore mitigating privacy risks arising from deletion notification distribution. In addition, the scalability challenges can be further mitigated by designing a distributed regulatory architecture composed of geographically partitioned regional centers, enabling load balancing and improved fault tolerance.

CRedit authorship contribution statement

Qipeng Song: Writing – review & editing, Methodology, Conceptualization. **Ruiyun Wang:** Writing – original draft, Software, Methodology. **Yue Li:** Supervision, Methodology, Funding acquisition, Conceptualization. **Yiheng Yan:** Software. **Xingyue Zhu:** Visualization. **Hui Li:** Funding acquisition, Conceptualization.

Acknowledgments

We would like to thank all the reviewers and editors for their very careful review of our manuscript. This work was supported in part by the National Natural Science Foundation of China (U24A20240), in part bin part by the Fundamental Research Funds for the Central Universities of Ministry of Education of China (ZYTS25071).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] A. Mantelero, The EU proposal for a general data protection regulation and the roots of the ‘right to be forgotten’, *Computer Law & Security Report* 29 (3) (2013) 229–235.
- [2] K. Myers, H. Lively, C. Andrews, New laws bring much tougher data protections: The California Consumer Privacy Act and the EU’s General Data Protection Regulation grant more rights to consumers and impose obligations on businesses, *Journal of Accountancy* 228 (5) (2019) 42–43.
- [3] J. Chen, J. Sun, Understanding the Chinese Data Security Law, *International Cybersecurity Law Review* 2 (2021) 209–221.

- [4] Data Protection Commission, The right to erasure (Articles 17 & 19 of the GDPR), 2022, [01-30-2024]. <https://www.dataprotection.ie/en/individuals/know-your-rights/right-erasure-articles-17-19-gdpr>.
- [5] J. Xiong, L. Chen, M.Z.A. Bhuiyan, C. Cao, M. Wang, E. Luo, X. Liu, A secure data deletion scheme for IoT devices through key derivation encryption and data analysis, *Future Generation Computer Systems* 111 (2020) 741–753.
- [6] J. Bartusek, D. Khurana, A. Poremba, Publicly-verifiable deletion via target-collapsing functions, in: *Proceedings of the Advances in Cryptology - CRYPTO 2023*, Springer, Cham, 2023, pp. 99–128.
- [7] H. Xu, T. Zhu, L. Zhang, W. Zhou, P.S. Yu, Machine unlearning: A survey, *ACM Computing Surveys* 56 (1) (2024) 36.
- [8] A. Thudij, H. Jia, I. Shumailov, N. Papernot, On the necessity of auditable algorithmic definitions for machine unlearning, in: *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*, USENIX Association, Berkeley, 2022, pp. 4007–4022.
- [9] J. Reardon, D. Basin, S. Capkun, SoK: Secure data deletion, in: 2013 IEEE symposium on security and privacy, IEEE, Piscataway, 2013, pp. 301–315.
- [10] B. Lee, K. Son, D. Won, S. Kim, Secure data deletion for USB flash memory, *Journal of Information Science and Engineering* 27 (3) (2011) 933–952.
- [11] Z. Mo, Y. Qiao, S. Chen, Two-party fine-grained assured deletion of outsourced data in cloud systems, in: *Proceedings of the 2014 IEEE 34th International Conference on Distributed Computing Systems*, IEEE, Piscataway, 2014, pp. 308–317.
- [12] J. Xiong, X. Liu, Z. Yao, J. Ma, Q. Li, K. Geng, P.S. Chen, A secure data self-destructing scheme in cloud computing, *IEEE Transactions on Cloud Computing* 2 (4) (2014) 448–458.
- [13] L. Yang, T. Wei, F. Zhang, J. Ma, SADUS: Secure data deletion in user space for mobile devices, *Computers & Security* 77 (2018) 612–626.
- [14] S. Gnatyuk, V. Kinzeravyy, T. Sapozhnik, I. Sopilko, N. Seilova, A. Hrytsak, Modern method and software tool for guaranteed data deletion in advanced big data systems, in: *Proceedings of the Advances in Artificial Systems for Medicine and Education II*, Springer, Cham, 2020, pp. 581–590.
- [15] N. Chen, B. Chen, Duplicates also matter! Towards secure deletion on flash-based storage media by removing duplicates, in: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ACM, New York, 2022, pp. 54–66.
- [16] L. Xue, Y. Yu, Y. Li, M.H. Au, X. Du, B. Yang, Efficient attribute-based encryption with attribute revocation for assured data deletion, *Information Sciences* 479 (2019) 640–650.
- [17] D. Zheng, L. Xue, C. Yu, Y. Li, Y. Yu, Toward assured data deletion in cloud storage, *IEEE Network* 34 (3) (2020) 101–107.
- [18] J. Ma, M. Wang, J. Xiong, Y. Hu, CP-ABE-based secure and verifiable data deletion in cloud, *Security and Communication Networks* 1 (2021) 8855341.
- [19] J. Tian, Z. Wang, Cloud data assured deletion scheme based on dynamic sliding window, *Peer-to-Peer Networking and Applications* 15 (4) (2022) 1817–1833.
- [20] S. Bauer, N.B. Priyantha, Secure data deletion for Linux file systems, in: *Proceedings of the 10th USENIX Security Symposium (USENIX Security 01)*, 2001.
- [21] W.A. Bhat, S.M.K. Quadri, RestFS: Secure data deletion using reliable & efficient stackable file system, in: *Proceedings of the 2012 IEEE 10th International Symposium on Applied Machine Intelligence and Informatics (SAMI)*, IEEE, Piscataway, 2012, pp. 457–462.
- [22] R.Z. Du, P.L. Shi, X.F. He, Cloud data assured deletion scheme based on overwrite verification, *Journal on Communications* 40 (1) (2018) 130–140.
- [23] C. Yang, Y. Liu, X. Tao, F. Zhao, Publicly verifiable and efficient fine-grained data deletion scheme in cloud computing, *IEEE Access* 8 (2020) 99393–99403.
- [24] C. Yang, X. Chen, Y. Xiang, Blockchain-based publicly verifiable data deletion scheme for cloud storage, *Journal of Network and Computer Applications* 103 (1) (2018) 185–193.
- [25] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, Y. Zhang, When machine unlearning jeopardizes privacy, in: *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, ACM, New York, 2021, pp. 896–911.
- [26] A. Gohatkar, A. Achille, A. Ravichandran, M. Polito, S. Soatto, Mixed-privacy forgetting in deep networks, in: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Piscataway, 2021, pp. 792–801.
- [27] J. Wang, S. Guo, X. Xie, H. Qi, Federated unlearning via class-discriminative pruning, in: *Proceedings of the ACM Web Conference 2022*, ACM, New York, 2022, pp. 622–632.
- [28] L. Bourtole, V. Chandrasekaran, C.A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot, Machine unlearning, in: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, Piscataway, 2021, pp. 141–159.
- [29] A. Sekhari, J. Acharya, G. Kamath, A.T. Suresh, Remember what you want to forget: Algorithms for machine unlearning, *Advances in Neural Information Processing Systems* 34 (2021) 18075–18086.
- [30] Y. Liu, M. Fan, C. Chen, X. Liu, Z. Ma, L. Wang, J. Ma, Backdoor defense with machine unlearning, in: *Proceedings of the IEEE INFOCOM 2022-IEEE conference on computer communications*, IEEE, Piscataway, 2022, pp. 280–289.
- [31] D.M. Sommer, L. Song, S. Wagh, P. Mittal, Towards probabilistic verification of machine unlearning, 2020. <https://doi.org/10.48550/arXiv.2003.04247>.
- [32] T. Eisenhofer, D. Riepel, V. Chandrasekaran, E. Ghosh, O. Ohrimenko, N. Papernot, Verifiable and provably secure machine unlearning, 2022. <https://doi.org/10.48550/arXiv.2210.09126>.
- [33] S. Constable, J. Van Bulck, X. Cheng, Y. Xiao, C. Xing, I. Alexandrovich, T. Kim, F. Piessens, M. Vij, M. Silberstein, AEX-notify: Thwarting precise single-stepping attacks through interrupt awareness for intel SGX enclaves, in: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, Berkeley, 2023, pp. 4051–4068.
- [34] A. Kogler, D. Gruss, M. Schwarz, Minefield: A software-only protection for SGX enclaves against DVFS attacks, in: *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*, USENIX Association, Berkeley, 2022, pp. 4147–4164.
- [35] S. Knight, H. Nguyen, N. Falkner, R. Bowden, M. Roughan, The Internet topology zoo, *IEEE Journal on Selected Areas in Communications* 29 (9) (2011) 1765–1775.

Qipeng Song received the B.S. degree in telecommunication engineering from Xidian University, Xi'an, China, in 2011, the M.S. and Ph.D. degrees in computer network and security from Télécom Bretagne, France, in 2014, and 2017. He is currently an Associate Professor with the School of Cyber Engineering, Xidian University. His research interests include SDN/NFV security, cloud security, and privacy protection.





Ruiyun Wang received the B.S. degree in School of Information Security from Xi'an University of Posts & Telecommunications, China, in 2023. She is currently studying in School of Cyber Engineering, Xidian University. Her research interests include information security and privacy protection.



Yue Li received the B.S. degree in microelectronics from Xidian University, Xi'an, China, in 2011, the M.S. degree in network engineering from Télécom Bretagne, France, in 2013 and the Ph.D. degree in Information and networks from Télécom ParisTech, France, in 2019. She is currently an Assistant Professor with the School of Cyber Engineering, Xidian University. Her research interests include next generation Internet architecture, AI security and privacy protection.



Yiheng Yan is currently a senior student at the School of Cyber Engineering, Xidian University. His research interests include information security and privacy protection.



Xingyue Zhu is currently a senior student at the School of Cyber Engineering, Xidian University. His research interests include information security and privacy protection.



Hui Li received the B.S. degree from Fudan University in 1990, the M.S., and Ph.D. degrees from Xidian University, in 1993 and 1998, respectively. Since June 2005, he has been a professor with the School of Cyber Engineering, Xidian University, Xi'an, Shaanxi, China. He has authored or coauthored more than 170 international academic research papers on information security and privacy preservation. His research interests include the areas of cryptography, wireless network security, information theory, and network coding. He is the chair of the ACM SIGSAC CHINA. He was the technique committee chair or the co-chair of several conferences.